how-to | Status: current | App v1.0.20 | Reviewed 2026-10-24

How-to Guides

Each guide gives a goal, prerequisites, steps, and a way to verify. Commands run from `Android/src/` unless stated.

#	Guide
1	Build and run the app locally
2	Configure Hugging Face OAuth
3	Add or change a model in the catalog
4	Test a catalog change without publishing
5	Write and test an Agent Skill
6	Add a new feature task
7	Add a native tool or intent for Agent Skills
8	Add a Mobile Actions function
9	Connect an MCP server
10	Import or side-load a model
11	Enable Firebase (Analytics and FCM)
12	Debug: logs, bug reports, benchmark

---

1. Build and run the app locally

Prerequisites: Android Studio with Android SDK platform 37 installed. A JDK 17+ for Gradle (CI uses Temurin 21). An Android 12+ device; a real device is recommended, since GPU/NPU acceleration and model sizes make emulators impractical.

```
        git clone https://github.com/google-ai-edge/gallery.git
        cd gallery/Android/src
        ./gradlew installDebug          # build + install debug APK
        # or: ./gradlew assembleRelease  # what CI builds (debug-signed, see 11-build-ci-
release)
```

Or open `Android/src` in Android Studio and **Run**.

Verify: "Edge Gallery" appears in the launcher, and the home screen lists tasks. Downloading gated Hugging Face models needs guide 2 first. Ungated models download without it.

2. Configure Hugging Face OAuth

Needed for gated models (e.g. Gemma) that require a license acceptance on Hugging Face.

1. Create an OAuth app on Hugging Face (docs). Choose a redirect URL such as `com.example.gallery://oauth2callback`, and include the `read-repos` scope.
2. Edit `gallery/common/ProjectConfig.kt`:

```
const val clientId = "<your client id>"
const val redirectUri = "com.example.gallery://oauth2callback"
```

3. Edit `Android/src/app/build.gradle.kts`. The value must be the scheme of `redirectUri`:

```
manifestPlaceholders["appAuthRedirectScheme"] = "com.example.gallery"
```

4. Rebuild. **Don't commit real values.** Keep them in a local, uncommitted change.

Verify: Downloading a gated model opens the Hugging Face login, returns to the app, and the download starts.

3. Add or change a model in the catalog

The app fetches `model_allowlists/<versionName with _>.json` from the `main` branch at runtime. **Edits reach users as soon as they're merged, without an app release.**

1. Open the allowlist for each app version you want to affect, e.g. `model_allowlists/1_0_20.json`.
2. Add an entry (see the field reference in 04-technical-design §1.2):

```
{
  "name": "Gemma3-1B-IT",
  "modelId": "litert-community/Gemma3-1B-IT",
  "modelFile": "gemma3-1b-it-int4.litertlm",
  "description": "Markdown shown on the model card.",
  "sizeInBytes": 584417280,
  "minDeviceMemoryInGb": 6,
  "commitHash": "42d538a932e8d5b12e6b3b455f5572560bd60b2c",
  "defaultConfig": { "topK": 64, "topP": 0.95, "temperature": 1.0, "maxTokens": 1024, "accelerators": "gpu,cpu" },
  "taskTypes": ["llm_chat", "llm_prompt_lab"],
  "bestForTaskTypes": ["llm_chat", "llm_prompt_lab"]
}
```

3. Rules:
   - `name` must be unique and contain no `/`.

- `commitHash` must be the exact Hugging Face revision.
- `sizeInBytes` must match the file size.
- `taskTypes` must use ids from `BuiltInTaskId`. `llm_test` is ignored.
- To retire a model, prefer `"disabled": true` over deleting the entry, so users who already downloaded it aren't surprised.

4. Check that the JSON is valid: `python3 -m json.tool model_allowlists/1_0_20.json > /dev/null`.

Verify: Use guide 4 before merging.

4. Test a catalog change without publishing

```
adb push model_allowlists/1_0_20.json /data/local/tmp/model_allowlist_test.json
# relaunch the app → a warning dialog confirms the local allowlist is in use
adb shell rm /data/local/tmp/model_allowlist_test.json   # revert
```

5. Write and test an Agent Skill

Full authoring reference: skills/README.md. Runtime details: 02-agents §5.

1. Create a kebab-case folder that contains a `SKILL.md` with `name` and `description` front matter and instructions.
2. For logic, add `scripts/index.html` that defines `window.ai_edge_gallery_get_result = async (data) => JSON.stringify({result: …})`. In `SKILL.md`, tell the model to call `run_js` with exact data fields.
3. Choose where the skill lives:

Target	Location
Built-in (ships in the APK)	`Android/src/app/src/main/assets/skills/<name>/` (12 skills). `skills/built-in/` publishes 8 of them. Keep both copies identical; `interactive-map/SKILL.md` already differs (see 14-known-issues).
Community	`skills/featured/<name>/` (deployed to GitHub Pages on merge to `main`)
Personal	Any HTTPS host (add from URL) or a local folder (import)

4. Test: Agent Skills, then Skill manager, then add the skill, then use the **Skill tester**, then ask a matching question. Watch `adb logcat | grep -E 'AGRunJsTool|AGGalleryWebView'`.

Verify: The chat shows "Calling JS script ..." progress and the final answer, or the image or webview card.

6. Add a new feature task

Use `customtasks/examplecustomtask/` as a template.

1. Create `gallery/customtasks/<feature>/`.
2. Implement `CustomTask` in `<Feature>Task.kt`:
   - `task = Task(id = "my_feature", label = …, category = Category.LLM, models = mutableListOf(...) or modelNames = listOf(...), description = …)`
   - `initializeModelFn`: load the model and set `model.instance`, then call `onDone("")`. On failure call `onDone("<error>")`.
   - `cleanUpModelFn`: close native resources, set `model.instance = null`, call `onDone()`.
   - `@Composable MainScreen(data: Any)`: cast to `CustomTaskData`, which gives you `modelManagerViewModel` and app-bar callbacks.
3. Register it:

```
        @Module @InstallIn(SingletonComponent::class)
        internal object MyFeatureTaskModule {
          @Provides @IntoSet fun provideTask(@ApplicationContext context: Context):
CustomTask = MyFeatureTask(context)
        }
```

4. Models come from either `task.models` (inline), or from the allowlist through `taskTypes: ["my_feature"]` / `task.modelNames`.
5. Optional: add the id to `BuiltInTaskId` and to `PREDEFINED_LLM_TASK_ORDER` to place it on the home screen.

Verify: The tile appears on the home screen. Opening it lists models, and selecting one runs `initializeModelFn`.

7. Add a native tool or intent for Agent Skills

Intent (callable through `run_intent`):

1. Add `MY_ACTION("my_action")` to `IntentAction` in `gallery/intents/IntentHandler.kt`.
2. Add a Moshi params data class and a `when` branch in `handleAction`. Return `"succeeded"`, `"failed"`, or a result string. Request permissions through the supplied `requestPermission` lambda.
3. Write or update a skill whose instructions call `run_intent` with `intent: "my_action"` and the JSON parameters.

Tool (always available to the model):

1. Implement `ToolDefinition` with a method annotated `@Tool(description = …)` and `@ToolParam` parameters. Return `Map<String, String>` with `status`.
2. Add it to `AgentToolsImpl.getAvailableTools()`, or call `agentTools.registerTool(tool)` at runtime.
3. Mention it in the system prompt template if the model needs routing guidance.

8. Add a Mobile Actions function

Follow Function_Calling_Guide.md. In short:

1. Add an `ActionType` and an `Action` subclass in `Actions.kt`.
2. Add a `@Tool` method in `MobileActionsTools.kt`.
3. Handle the action in `MobileActionsViewModel.performAction`.

The 270M model was fine-tuned on the existing functions, so new functions usually need a re-tuned model.

9. Connect an MCP server

Follow mcp/README.md:

1. Run the server in StreamableHTTP mode. Use `npx -y supergateway --stdio '<cmd>' --outputTransport streamableHttp` for stdio-only servers.
2. Expose it over HTTPS, e.g. `cloudflared tunnel --url http://localhost:8000`.
3. In the app: Agent Skills, MCP chip, Manage, **+**, Add from URL. Enter `https://…/mcp` and optionally a header (e.g. `X-Goog-Api-Key`).
4. Enable the tools you want, then try a prompt.

10. Import or side-load a model

- **In-app import:** Model manager, then Import. Pick a `.litertlm` (or `.task`) file and set its capabilities (accelerators, max tokens, image/audio/thinking). It is copied to `<modelsDir>/__imports/`. Use the "Test" chat (`llm_test`).
- **From Hugging Face:** Model manager, then Explore (search/sort) or Import from Hugging Face URL.
- **adb (large files):** `adb push model.litertlm /data/local/tmp/` and import from there, or reference it with `localModelFilePathOverride` in a local test allowlist (guide 4).

11. Enable Firebase (Analytics and FCM)

The open-source build has Firebase turned off. To turn it on:

1. Add your `google-services.json` to `Android/src/app/` (keep it out of git).
2. In `Android/src/app/build.gradle.kts`, change `alias(libs.plugins.google.services) apply false` to `apply true`.
3. Rebuild. Events are listed in 04-technical-design §10. Users can opt out in Settings.

12. Debug: logs, bug reports, benchmark

Need	Do
App logs	`adb logcat
Download issues	Filter `AGDownloadWorker`, `AGModelManagerViewModel`
Agent / tool issues	Filter `AGDefaultAgentRuntimeExecutor`, `AGRunJsTool`, `AGRunMcpTool`, `AGRunIntentTool`, `AGSkillManager`
Full bug report	Bug_Reporting_Guide.md (`adb bugreport`)
Performance	Model list, then Benchmark (LiteRT-LM models). Results are stored and comparable.